

ADR 0002 — Criptografia ponta-a-ponta (E2EE) das mensagens táticas

- **Status:** Aceita (Fase 5 — issue #64). Entrega faseada — ver "Escopo de entrega".
- **Data:** 2026-07-13
- **Regras relacionadas:** [mqtt-multitenant.md](#), [timezone-dates.md](#)

Contexto

O Cerberus coordena operações da Polícia Federal. O conteúdo das mensagens táticas (diretivas da central, reportes de campo) é sensível: mesmo com **TLS no transporte** e **isolamento multitenant na API** (ADR 0001), o servidor e o broker MQTT ainda veriam o texto em claro em trânsito e em repouso. A Fase 5 pede **E2EE**: só os dispositivos das partes leem o conteúdo; servidor e broker manuseiam apenas material opaco.

Restrições que moldam a decisão:

- O mesmo código de cripto precisa rodar em **Node** (API/testes), **navegador** (dashboard) e **React Native/Hermes** (app). Web Crypto não é uniforme nos três; primitivas nativas divergem.
- O app móvel fica **fora dos workspaces npm** (particularidade do Metro) — não consome [@cerberus/shared](#) como pacote; mantém um **espelho** local dos contratos.
- Um broadcast é **1→N** (central → todos os agentes da operação): não dá para cifrar par-a-par ingênuo sem re-cifrar a mensagem inteira por destinatário.

Decisão

Primitiva: NaCl (tweetnacl), não AES-256

Adotamos **NaCl** via [tweetnacl](#) + [tweetnacl-util](#) (JS puro, idêntico em Node/navegador/Hermes), em vez do "AES-256" esboçado na especificação inicial. Identidade é um par **X25519** ([nacl.box](#) — Curve25519 + XSalsa20-Poly1305); o conteúdo usa [nacl.secretbox](#) (XSalsa20-Poly1305). [box](#) já provê **autenticação** (o destinatário confirma que veio de quem detém a chave privada do remetente), o que AES-CBC/GCM cru não daria sem MAC/gestão de IV manual.

No app, o [tweetnacl](#) exige [crypto.getRandomValues](#); o polyfill [react-native-get-random-values](#) é importado em [src/polyfills.ts](#) **antes** de qualquer módulo de cripto (CSPRNG do gerador de chaves/nonces).

É E2EE de verdade

A chave **privada nunca sai do dispositivo/cliente**. Só a **pública** é registrada no servidor. O servidor e o broker **nunca** recebem o texto em claro nem a chave privada — apenas o envelope cifrado. Comprometer a API/Mongo/broker **não** revela o conteúdo das mensagens.

Envelope por destinatário (1→N)

A mensagem é cifrada **uma vez** com uma chave simétrica aleatória **K**; **K** é **embrulhada por destinatário** com **box**. Formato (helpers em `packages/shared/src/e2ee.ts`, espelhado em `apps/mobile/src/shared/e2ee.ts`), serializado como **string JSON base64** e transportado no campo **ciphertext** da mensagem:

```
{
  "v": 1,
  "ct": "<secretbox(plaintext, n0, K)>", // conteúdo cifrado (base64)
  "n0": "<nonce da mensagem>",        // base64
  "spk": "<chave pública do remetente>", // base64
  "envs": [                             // K embrulhada, uma entrada por
destinatário
    { "rid": "AG-0456", "ek": "<box(K, n, pubDest, privRemet)>", "n": "<nonce>"
    }
  ]
}
```

Decifra: o destinatário acha seu **env** por **rid**, faz **K = box.open(ek, n, spk, minhaPriv)** e **plaintext = secretbox.open(ct, n0, K).openMessage** devolve **null** (sem lançar) se a mensagem não for para ele, se a chave não bater, ou se **não for um envelope** (ex.: uma mensagem de sistema em claro no mesmo canal) — ver "Convivência com texto em claro".

Identidade do destinatário (rid): é o **agentId** do agente ou o **userId** do admin (**agentId ?? userId**). O diretório expõe esse **id**; o cliente decifra usando o **seu próprio agentId ?? userId** — os dois lados precisam concordar.

Armazenamento de chaves

Onde	Privada	Pública
App (agente)	<code>expo-secure-store</code> (Keystore/Keychain), por <code>userId</code>	registrada na API
Dashboard (operador)	<code>localStorage</code> por <code>userId</code> — MVP (ver limitações)	registrada na API
API	— (nunca vê a privada)	<code>User.publicKey</code> (base64)

O provisionamento é **automático no login**: se não há par local, gera → guarda a privada → registra a pública (**PUT /auth/public-key**). Falha de rede **não bloqueia** o login (re-tenta no próximo). A privada nunca trafega.

Diretório de chaves (API)

- PUT /auth/public-key** (autenticado) — o portador registra/atualiza a **própria** pública (validada como base64 de 32 bytes).

- `GET /operations/:id/keys` (autenticado + `assertOperationScope`) — devolve `{ id, userId, role, agentId?, publicKey }` dos membros **da operação** que já registraram chave. Escopado por operação como qualquer rota multitenant (ADR 0001). Agentes também leem (habilita o fluxo agente→central).

Convivência com mensagens de sistema em claro

Os **alertas de geofence** são publicados pela ponte da API no mesmo canal

`operacao/{opId}/broadcast`, em claro (`{ type: 'alert', text }`), pois são gerados no servidor e não têm um remetente humano com chave. O consumidor decide por presença de campo: **ciphertext presente → decifra; senão text → exibe como-está**. `openMessage` devolvendo `null` para não-envelopes garante que texto de sistema nunca é confundido com conteúdo cifrado.

Escopo de entrega (faseado dentro da Fase 5)

- **Entregue (PR #65 fundação + PR #66 broadcast):** pares de chaves + diretório; **broadcast central→agentes E2EE** (dashboard cifra pelo diretório; API persiste/publica só `ciphertext`; `serialize()` devolve `ciphertext`; app decifra no serviço MQTT).
- **Restante da Fase 5:** texto **agente→central** cifrado (`POST /messages`), **UI de recepção/histórico cifrado no dashboard** (decifrar broadcast/texto), e **E2EE de mídia** (binário no GridFS). Enquanto não entregue, `POST /messages` (texto) segue em claro.

Referência de implementação (mapa técnico completo)

Dependências e versões

- `tweetnacl` + `tweetnacl-util` — em `packages/shared` (usado por API + dashboard via `@cerberus/shared`) e em `apps/mobile` (fora dos workspaces; instalado à parte).
- `react-native-get-random-values@~1.11.0` — **apenas no app**. Fixado na linha **1.x** de propósito: a **2.0.0** exige `react-native >= 0.81` e o app está em **0.76.5** (conflito ERESOLVE). Fornece `crypto.getRandomValues` para o CSPRNG do `tweetnacl`.

Helpers compartilhados — `packages/shared/src/e2ee.ts`

(Espelhado **verbatim** em `apps/mobile/src/shared/e2ee.ts` — manter em sincronia, como `contracts.ts`.)

Símbolo	Assinatura	Papel
<code>E2eeKeyPair</code>	<code>{ publicKey: string; secretKey: string }</code>	par X25519 em base64
<code>generateKeyPair()</code>	<code>() => E2eeKeyPair</code>	gera o par (<code>nacl.box.keyPair</code>)
<code>publicFromSecret(sk)</code>	<code>(string) => string</code>	deriva a pública da secreta (registra sem guardar as duas)
<code>E2eeRecipient</code>	<code>{ id: string; publicKey: string }</code>	destinatário do envelope

Símbolo	Assinatura	Papel
<code>sealMessage(txt, sk, recips)</code>	<code>(string, string, E2eeRecipient[]) => string</code>	cifra 1→N; devolve o envelope (string)
<code>openMessage(ct, myId, sk)</code>	<code>(string, string, string) => string null</code>	decifra ou <code>null</code> (não-destinatário / chave errada / não-envelope)

`KeyDirectoryEntry ({ id, userId, role, agentId?, publicKey })` vive em `packages/shared/src/schemas.ts`, junto de `publicKeyRegistrationSchema ({ publicKey: z.string().regex(/^([A-Za-z0-9+]{43})=$/ ) }` — base64 de exatamente 32 bytes).

Superfície de API

Rota	Papel	Corpo / Resposta	Notas
<code>PUT /auth/public-key</code>	autenticado	<code>{ publicKey } → { publicKey }</code>	grava <code>User.publicKey</code> do portador
<code>GET /operations/:id/keys</code>	autenticado + escopo	→ <code>KeyDirectoryEntry[]</code>	só membros da operação com chave; <code>id = agentId ?? userId</code>
<code>POST /operations/:id/broadcast</code>	<code>requireRole(ADMIN)</code> + escopo	<code>{ ciphertext } (≤ 500 000 chars) → mensagem</code>	persiste/publica só <code>ciphertext</code> (sem <code>text</code>); tipo <code>BROADCAST</code>

- `User.publicKey: { type: String }` em `apps/api/src/models/index.ts` (a `MessageModel` já tinha `ciphertext: String`, antes reservado).
- `broadcastSchema` (local em `modules/messages/routes.ts`) limita o envelope a **500 000** chars — ele cresce com o nº de destinatários; o limite é folgado, não o de texto (4096).
- `serialize()` em `modules/messages/routes.ts` foi corrigido para devolver `ciphertext`** — antes o descartava, o que tornaria o histórico cifrado indecifrável na leitura.
- A ponte MQTT **não re-ingere** o broadcast: ela assina `operacao/+/agente/+/#` (canal do agente), e o broadcast sai em `operacao/{opId}/broadcast` — a persistência ocorre **só** na rota, sem duplicação.

Cliente — dashboard

- `apps/dashboard/src/lib/e2ee.ts: ensureKeyPair(userId), getSecretKey(userId), clearKeys(userId), provisionKeys(userId)`. Privada em `localStorage` sob a chave `cerberus_e2ee_sk:<userId>`.
- `provisionKeys` é chamado no **login** (`app/login/page.tsx`), com `.catch` que só avisa (não bloqueia).

- `api.registerPublicKey(publicKey)` e `api.operationKeys(operationId)` em `lib/api.ts`.
- `api.broadcast(operationId, ciphertext)` — assinatura mudou de `text` para `ciphertext`.
- `sendBroadcast()` (`operations/[id]/live/page.tsx`): busca o diretório, monta `recipients` com **todas** as entradas (inclui o próprio admin — habilita auto-decifra futura), mas **avisa e aborta** se nenhuma entrada tem `role === 'agente'`; cifra com `sealMessage` e envia o `ciphertext`.

Cliente — app (agente)

- `apps/mobile/src/services/keys.ts`: `ensureKeyPair(userId)`, `getSecretKey(userId)`, `provisionKeys({ userId, token })`. Privada em `SecureStore` sob a chave `cerberus_e2ee_sk_<userId>` — nota: `SecureStore` só aceita `[A-Za-z0-9._-]`, por isso `_` (não : como no dashboard).
- `provisionKeys` é chamado no `login()` (`services/auth.ts`), com `.catch` que só avisa.
- Decifra **no serviço MQTT** (`services/mqtt.ts`): `subscribeBroadcast(operationId, identity, listener)` recebe `identity = { myId, secretKey }`; `handleIncoming` chama `resolveText` → `openMessage` para `ciphertext` ou repassa `text` em claro. `OperationScreen` carrega a `secretKey` (assíncrono, `SecureStore`) antes de assinar, com `myId = agentId ?? userId`.

Consequências

- **Verificado por teste** (`apps/api/src/modules/routes.test.ts`): round-trip pelo endpoint prova que o servidor **não persiste nem retorna texto em claro** (`JSON.stringify(history)` não contém a diretiva) e que **só o destinatário** decifra; round-trip unitário dos helpers em `packages/shared/src/e2ee.test.ts` (inclui "não vaza texto", "não-destinatário → null").
- **Invariante para código novo de mensagens**: cifrar **no cliente**; a API **nunca** grava texto em claro de conteúdo E2EE; `serialize()` **deve** propagar `ciphertext` (senão o histórico fica indecifrável); toda rota escopada mantém `assertOperationScope`.
- **Nota operacional**: o 1º broadcast cifrado exige que o operador **e** ao menos um agente tenham feito login após o deploy (o login provisiona as chaves). Sem agente com chave, o dashboard avisa em vez de enviar às cegas.
- **Limitações / trabalho futuro de endurecimento**:
 - Dashboard guarda a privada em `localStorage` (estação confiável, MVP). Endurecer com passphrase (derivação) ou WebCrypto não-extraível.
 - O remetente é confiado pelo `spk` embutido no envelope + `senderId` que a API define; **não** há verificação do `spk` contra o diretório ainda (defesa contra troca de chave). Adicionar verificação `spk == diretório[senderId].publicKey` no cliente.
 - Sem rotação/revogação de chave nem histórico multi-chave; re-registrar uma pública nova torna ilegíveis os envelopes antigos daquele destinatário.